

Resonator-Factored Hierarchical Hypervector Embeddings for Compact Structured Memory and Faster Factor-Aligned Computation:

General Theory and Application to Columnar Causal Networks and Neural-Symbolic Associative Memory

Ben Goertzel

June 3, 2026

Abstract

We describe a family of resonator-decoded hierarchical hypervector codes for approximately factorized data and learned latent structures. We then show that, when a neural or algorithmic system exposes a bounded-load factor graph for a specified task family, vector-symbolic or hyperdimensional computing (HDC) can provide a compact, compositional, queryable cache of those factors.

The dimension required for reliable local queries depends on local crosstalk load, effective codebook size, cleanup margin, query depth, and the number of supported queries, but not directly on the raw number of pixels, frames, tokens, or leaves. We make this precise using ordered roles, finite or factorized level codebooks, query-limited capacity bounds, a task-specific factorization defect, and a conditional resonator margin theorem.

Next, we suggest to turn the HDC layer from a passive compression stage into an active guide for representation learning. The HDC capacity law tells the upstream learner what kind of factorization to seek: reusable, role-stable, locally decodable, low-load, high-margin factors with little task-relevant information left in the residual. We formulate HDC-admissibility diagnostics and losses for effective local load, cleanup margin, confusable codebook size, product recomposition, role consistency, residual predictive information, and intervention locality. This idea applies beyond DNNs to any structured algorithm whose internal variables can be aligned with HDC roles and codebooks.

We also show how this same machinery can be combined with Hierarchical Modular Hypervectors (HMH) and PRIMUS-style world modeling to yield resonant modular compositional episodic memory. In this variant, an episode is stored not as a monolithic trace but as a typed, role-filler, evidence-anchored factor graph: actors, objects, places, times, goals, affective and social context, causal links, affordance trials, outcomes, skills, and perceptual pointers are bound into ontology-aligned modules. Resonator cleanup then supports partial structured recall, slot completion, counterfactual factor swaps, schema consolidation, and neural judgment over densified HMH traces in a way that respects the factorization of the remembered episode.

Finally, we instantiate the framework in Hierarchical Bayesian Causal Modular Learning (HBCML) and Columnar Bayesian Causal Coding (ColBaC), a two-level columnar continual-learning architecture. In ColBaC, sparse Bayesian support selection, internal column certificates, hard kernels, adaptable shells, typed microcolumns, and local counterfactual teachers give an especially direct substrate for HDC-guided factorization. ColBaC learns and protects causal columns; the HDC layer makes their reusable contents addressable, compositional, auditable, and capacity-aware.

AI Acknowledgement Lots of help from GPT-5.5-Pro, Claude Opus 4.7 and Gemini Pro

Contents

1	Introduction	3
2	Vector-symbolic algebra	5
3	Learned factor graphs	7
3.1	From raw data to a quantized latent graph	7
3.2	Supported query model	8
4	Encoder and query algorithms	8
4.1	Hierarchical code	8
4.2	Path descent	9
4.3	Resonator factorization	9
4.4	Structured queries	10
5	Theory	10
5.1	Cleanup capacity	11
5.2	Conditional resonator theorem	12
5.3	Task preservation for DNN latents	13
5.4	Storage accounting	14
5.5	Speed and baseline accounting	14
5.6	Compression-compute accounting	14
6	Scalability objections as design criteria	15
6.1	Effective rather than enumerative subtree dictionaries	15
6.2	Binding must be fixed or quasi-isometric	15
6.3	Resonator search cost must be charged	16
6.4	The factorization defect is a test, not a rescue clause	16
6.5	Admissibility checklist	16
7	HDC-guided factorization learning	16
7.1	Representation families that can supply factors	17
7.2	HDC-admissible factorization	17
7.3	Measuring factorization error during learning	18
7.4	Local load loss	18
7.5	Cleanup margin and confusability losses	18
7.6	Product recomposition and resonator identifiability	19
7.7	Role stability	19
7.8	Residual synergy test	19
7.9	Intervention locality for causal coding	19
7.10	Total HDC-guided objective	20
7.11	Live diagnostics	20
8	Resonant modular compositional episodic memory and world modeling	20
8.1	Review: HMM and PRIMUS-style world modeling	20
8.2	A factorized episode model	21
8.3	Resonant partial recall and slot completion	22
8.4	Factor-aware neural judgment	22

8.5	Episodic-semantic consolidation	23
8.6	Capacity criteria for episodic memory	23
8.7	Operational workflow	24
9	Instantiation in HBCML and ColBaC	24
9.1	HBCML background	25
9.2	ColBaC background	25
9.3	Why HDC fits ColBaC	26
9.4	HDC certificates	26
9.5	HDC-guided shell hygiene	26
9.6	The bridge microcolumn	27
9.7	Episodic support memory	27
9.8	Option memory for RL and agentic systems	27
9.9	A ColBaC-HDC error decomposition	28
9.10	Why this is more general than ColBaC	28
10	Experimental program	28
10.1	Synthetic calibration	28
10.2	DNN latent experiments	29
10.3	HDC-guided learning experiments	29
10.4	R-HMH episodic memory and world-modeling experiments	30
10.5	ColBaC-HDC experiments	30
11	Application regimes	30
12	Discussion	31
13	Conclusion	32

1 Introduction

Images, video, language, program traces, relational records, and agent memories are high-dimensional objects. They are not, in general, losslessly compressible into one fixed-width vector. Yet most useful computations over them never require full reconstruction. They require *local* access to object states, roles, events, spans, mechanisms, retrieved passages, syntax subtrees, options, or other reusable latent factors. This paper studies a representation layer for precisely that regime: a compact structured memory that one can store cheaply, query locally, and—when the memory’s factorization is aligned with the host model’s own factor structure—compute over directly, rather than re-deriving it from raw data.

The intended pipeline is

$$x \xrightarrow{E_\theta} z(x) \xrightarrow{F_\psi} G_x \xrightarrow{\text{Enc}} H_x \in \mathbb{H}_D. \quad (1)$$

Here x is raw data; E_θ is a perceptual, linguistic, causal, or algorithmic encoder; $z(x)$ is its full internal state; F_ψ extracts a roughly factorized latent graph G_x ; and H_x is a fixed-width vector-symbolic code, or hyperdimensional embedding, of that graph. The HDC layer is not asked to solve perception or language modeling from scratch. It is asked to store and query a structured latent graph that another system has already learned or constructed, in a form whose algebra matches the structure of the host computation.

The central distinction is between all-information reconstruction and query-faithful representation. A code that answered every possible leaf query for an N -leaf object with uniformly high probability would permit reconstruction of the whole object, and must pay the usual entropy cost. A code that supports a fixed or slowly growing family of local, role-bound, or homomorphic queries can be far smaller. The dimension that controls reliability is set by the local interaction load and the effective number of confusable states at each cleanup step, not by raw object size alone.

This is also where the computational payoff comes from. When the data is factorized in a way that matches the factor structure the host model already uses, a query can be answered, and a reusable factor retrieved or recombined, by operating on the compact code instead of scanning the raw object or re-running the encoder. Two cost regimes matter. First, resonator factorization recovers an unknown product of factors by iterated slot-wise cleanup at cost proportional to $\sum_f |\mathcal{C}_f|$, rather than by brute-force search over the product space $\prod_f |\mathcal{C}_f|$; this is the exponential-to-linear win, conditional on a measurable margin. Second, a local role-bound query touches only the few factors on its path, so its cost scales with query depth and local load rather than with object size, and it can replace a raw or full-latent scan, dense attention over stored contexts, or energy-intensive neural recomputation. We are deliberate about the limits of this claim: a single local HDC lookup does not asymptotically beat a well-engineered indexed tree, and when the encoder E_θ must run at query time its compute is charged separately from the cached code. The speedups are real, but they are regime-specific and live in the memory, query, and reuse layer—and, in columnar architectures, in the operation of the columns themselves.

The two error sources separate cleanly:

$$\mathcal{L}_{\text{HDC}} - \mathcal{L}_{\text{full}} = \underbrace{\mathcal{L}_{\text{fact}} - \mathcal{L}_{\text{full}}}_{\text{factorization defect}} + \underbrace{\mathcal{L}_{\text{HDC}} - \mathcal{L}_{\text{fact}}}_{\text{HDC coding error}}. \quad (2)$$

The HDC theory controls the second term through dimension, margin, and local load. It does not control the first. If the host system relies on holistic, distributed synergies that no bounded-load factor graph can approximate for the chosen task family, the defect term stays large even as $D \rightarrow \infty$. This is not a loophole; it is the main empirical diagnostic for whether a structured hypervector memory is the right tool at all.

A second theme is that the HDC layer need not be a passive cache placed after the host model. The same capacity law that says when a code is reliable also says which factorizations are cheap to store and fast to query: reusable, role-stable, locally decodable, low-load, high-margin factors that leave little task-relevant information in the residual. Read in that direction, the capacity law becomes a learning objective. The host learner is pressed not merely to compress, but to factorize in a way that supports cheap binding, bundling, cleanup, resonator factorization, and role-conditioned query. We formulate the corresponding admissibility diagnostics and losses, and note that they apply to any structured algorithm—not only deep nets—whose internal variables can be aligned with HDC roles and codebooks.

This same idea also gives a useful way to reinterpret episodic memory. Episodic traces in agents are not merely bags of percepts, and not merely timestamps on raw observations. They typically have a semi-factorized organization: who did what to whom, where, when, for what reason, under what affordance, social, normative, or causal constraints, with what evidence, and with what outcome. Hierarchical Modular Hypervectors (HMH) already provide a typed modular role-filler representation for such structures, while PRIMUS-style world modeling uses HMH as a structured associative regime between symbolic atoms and dense neural tensors. Adding resonator cleanup to this substrate yields what we will call resonant modular compositional episodic memory: an episode can be stored as an HMH-like factor graph, retrieved by partial structured cues, completed by slot-wise resonator cleanup, densified for neural judgment, and consolidated into semantic schemas,

affordance templates, social scripts, or reusable skills. This is not a separate application bolted onto the paper’s theory; it is a direct case of the general query-limited story, with episodes playing the role of structured latent graphs and episodic questions playing the role of supported local queries.

This perspective is especially natural in columnar causal architectures, where we develop it concretely. Hierarchical Bayesian Causal Modular Learning (HBCML) decomposes continual learning into a sparse top-level controller that selects structurally restricted components and an internal probabilistic process that separates reusable causal structure from task-local residue. Columnar Bayesian Causal Coding (ColBaC) is a concrete columnar instance with hard kernels, adaptable shells, typed microcolumns, internal certificates, and local one-swap teachers. These architectures already expose the objects a structured memory needs—supports, columns, shells, motifs, certificates, options—so HDC can supply the algebraic language in which their reusable contents become addressable, compositional, auditable, and capacity-aware. Here the cache-level speedups and the cost of operating the network largely coincide, which is why ColBaC is our principal application rather than an afterthought.

The paper makes four design commitments. First, descent through a hierarchy uses explicit level dictionaries or factored dictionaries of possible child states; a dictionary of all formal subtrees is intractably large, so what matters is the effective metric entropy of the task-relevant learned states. Second, binding is either a fixed vector-symbolic algebra, such as MAP or phasor HRR, or a learned operation carrying an explicit quasi-isometry penalty. Third, resonator decoding is claimed only under a measurable margin condition, with all search overheads charged. Fourth, speedup is always stated against a named baseline: an HDC local query does not asymptotically beat a conventional indexed tree lookup, but it can beat raw scans, full latent scans, dense attention over stored contexts, brute-force factor search, or neural recomputation in the appropriate regime.

Contributions. The paper contributes: (i) a query-limited theory of hierarchical resonator HDC codes with explicit level codebooks, ordered roles, conditional resonator margins, a task-specific factorization defect, and honest storage and compute accounting; (ii) a compute model that ties compression to query speed and identifies the regimes in which factor-aligned hypervector computation is genuinely cheaper than scanning, dense attention, brute-force factor search, or recomputation; (iii) a set of scalability criteria that turn common objections into measurable quantities; (iv) an HDC-guided learning objective that pushes a host DNN or structured algorithm toward capacity-favorable factorizations; (v) a resonant HMM construction for episodic memory and PRIMUS-style world modeling, showing how factorized episodes can be retrieved, completed, densified, judged, and consolidated while preserving their role-filler structure; and (vi) a detailed instantiation in ColBaC/HBCML showing how HDC requirements can shape column selection, internal certificates, shell hygiene, support memory, and causal modularity.

2 Vector-symbolic algebra

The basic idea of a vector-symbolic architecture is that a single long vector can carry structured, symbol-like content, with a small algebra of operations that mirror the operations one would perform on symbols. Atoms play the role of distinct symbols; because they are random and high-dimensional, distinct atoms are nearly orthogonal, and this near-orthogonality is what lets many of them coexist in one vector without colliding.

Let $\mathbb{H}_D$ be a D -dimensional hypervector space. Examples include bipolar MAP vectors $\{-1, +1\}^D$ with elementwise multiplication, complex phasor HRR vectors with elementwise phase multiplica-

tion, or normalized real vectors with an approximately unitary binding operation. We use four operations.

- Binding $u \otimes v$ composes two vectors; intuitively it forms an association or key–value tag. For atom a , binding by a is approximately an isometry and has an inverse $a^\dagger$ satisfying $a^\dagger \otimes a = \mathbf{1}$. The bound vector is dissimilar to both of its inputs, so binding hides content; unbinding by the inverse, $a^\dagger \otimes (a \otimes v) = v$, recovers the partner.
- Bundling $u \oplus v$ superposes vectors; intuitively it forms an unordered set or memory trace. In most implementations this is ordinary addition followed by an optional normalization or quantization. Unlike binding, a bundle stays similar to each of its summands, so set membership can be tested by similarity, at the cost of crosstalk that grows with the number of superposed terms.
- Permutation ρ or depth-specific roles encode order. We do not rely on commutativity to distinguish paths: applying a permutation (or using a position-specific role atom) makes the contribution of one slot or path nearly independent of the others, so that, for example, (r_1, r_2) and (r_2, r_1) are not conflated.
- Cleanup maps a noisy vector to a nearby codebook element; it is the error-correction step that pulls an approximate estimate back onto a legal symbol.

A finite codebook is $\mathcal{C} = \{c_1, \dots, c_M\} \subset \mathbb{H}_D$, also viewed as a $D \times M$ matrix. Soft and hard cleanup are

$$\text{cleanup}_{\mathcal{C}}(z) = \mathcal{C} \text{softmax}(\beta \mathcal{C}^\top z), \quad \widehat{\text{cleanup}}_{\mathcal{C}}(z) = \arg \max_{c \in \mathcal{C}} \langle c, z \rangle. \quad (3)$$

Hard cleanup is nearest-neighbor decoding against the codebook; soft cleanup is its smoothed, differentiable relaxation, a softmax-weighted blend of codebook entries that sharpens toward the hard rule as $\beta \rightarrow \infty$. Soft cleanup is useful for differentiable or iterative updates; final readout uses the hard rule.

For soft intermediate states we also need a projection back to the algebraic state space before inversion:

$$\text{Proj}_{\mathbb{H}}(z) = \begin{cases} \text{sign}(z), & \text{bipolar MAP,} \\ z/|z|, & \text{phasor HRR, componentwise when } z_i \neq 0, \\ z/\|z\|, & \text{unit-sphere real model.} \end{cases} \quad (4)$$

A soft, softmax-weighted blend of atoms is not itself a legal element of the binding algebra, so unbinding it directly is not well defined; the projection snaps the estimate back onto the manifold (signs, unit phases, or unit norm) where binding inverses behave properly. Algorithmic inversion is therefore applied to $\text{Proj}_{\mathbb{H}}(z)$, not to an arbitrary convex combination of atoms.

Assumption 1 (Random atoms and concentration). Atoms are independent normalized random hypervectors with $\|a\|^2 = D$, $\mathbb{E}\langle a, a' \rangle = 0$ for independent $a \neq a'$, and sub-Gaussian normalized overlap: for some constant $c_0 > 0$,

$$\mathbb{P}(|\langle a, a' \rangle| \geq tD) \leq 2 \exp(-c_0 D t^2), \quad t \in (0, 1]. \quad (5)$$

Binding by an independent atom preserves these concentration bounds.

Assumption 2 (Isometric binding). For every atom a , $\langle a \otimes u, a \otimes v \rangle = \langle u, v \rangle$, and binding distributes over bundling before normalization. This holds exactly in common idealized MAP and Fourier-domain HRR models and approximately in many learned VSA layers.

Ordering convention. A commutative binding algebra cannot by itself distinguish the path (r_1, r_2) from (r_2, r_1) . We therefore use either depth-specific role atoms $r_{t,i}$ or role atoms composed with a permutation, e.g. $r_i \otimes \rho(H_{\text{child}})$. The theory below assumes that two distinct ordered paths have independent or near-independent role products.

3 Learned factor graphs

3.1 From raw data to a quantized latent graph

Let $x \in \mathcal{X}$ be a raw datum: an image, video clip, audio segment, document, program, molecule, or relational record. A neural encoder E_θ produces a latent state $z(x)$. A factorizing module F_ψ maps $z(x)$ to a finite graph or tree

$$G_x = (V_x, E_x), \quad (6)$$

with maximum depth L and maximum local branching b_t at level t . The graph may be a true tree, a tree decomposition of a graph, a scene graph, an object track hierarchy, a parse hierarchy, a multiscale patch hierarchy, or a causal state graph unfolded over time. The theory is simplest for trees. DAGs can be handled by duplicating shared nodes, using junction trees, or adding explicit cross-edge roles and treating those edges as additional local load.

Each node $\nu \in V_x$ has a latent state z_ν . At level $t(\nu)$, a quantizer or dictionary learner assigns z_ν to a finite prototype codebook

$$\mathcal{K}_t = \{k_{t,1}, \dots, k_{t,M_t}\} \subset \mathbb{H}_D. \quad (7)$$

The prototype may be an atom, a normalized subtree prototype, or a product of factor atoms

$$u_\nu = c_{\nu,1} \otimes \dots \otimes c_{\nu,F_t}, \quad c_{\nu,f} \in \mathcal{C}_{t,f}. \quad (8)$$

In DNN applications the codebooks $\mathcal{K}_t$ and $\mathcal{C}_{t,f}$ can be produced by VQ layers, product quantization, slot prototypes, sparse autoencoder dictionaries, object-state prototypes, or causal-state prototypes. The codebook sizes M_t are part of the theorem. If M_t grows exponentially with object size, then the HDC dimension must grow as well; the dimension is independent of raw size only when the learned latent uses reusable local prototypes.

Definition 1 (Neural factorization defect). Fix a task or query family $\mathcal{Q}$ and a loss d_q for each $q \in \mathcal{Q}$. The factorization defect of the learned graph family $\mathcal{G} = \{G_x\}$ relative to the full neural state $z(x) = E_\theta(x)$ is

$$\delta_{\mathcal{Q}} = \sup_{q \in \mathcal{Q}} \mathbb{E}_x [d_q(q(z(x)), q(G_x))]. \quad (9)$$

The defect includes quantization error, missing nonlocal interactions, and any information in $z(x)$ that the factor graph does not preserve for the query family. It is not assumed to be an energy ratio and need not be the same for all tasks.

This definition is deliberately task-specific. A video factor graph may have low defect for object tracking and high defect for photorealistic frame reconstruction. A document factor graph may have low defect for entity retrieval and high defect for open-ended generation. A language-model sparse feature graph may have low defect for a probing task and high defect for full next-token prediction.

3.2 Supported query model

Let $\mathcal{Q}_{\text{sup}}$ be the family of queries the code is designed to answer. A query may specify one or more root-to-node paths, a set of roles, a factor slot to recover, or a summary readout. Let

$$q_{\max} = |\mathcal{Q}_{\text{sup}}|, \quad \ell_{\max} = \max_{q \in \mathcal{Q}_{\text{sup}}} \text{depth touched by } q. \quad (10)$$

The local capacity theorem below is for a fixed supported query family. The queries can be chosen after seeing the training distribution and codebooks, but not adversarially after seeing the random atom draw unless the margin conditions are verified for that draw.

Remark 1 (Why the query parameter is necessary). If a constant-dimensional code could answer every one of N leaf queries with high probability independent of N , then querying all leaves would reconstruct the object. Thus any theorem claiming dimension independent of N must be query-limited, distributional, approximate, or nonuniform. We make this explicit through $q_{\max}$ and through the all-path lower bound in Theorem 2.

4 Encoder and query algorithms

4.1 Hierarchical code

For a node ν at level t , let $u_\nu \in \mathcal{K}_t$ be its quantized local state, possibly factored as in Eq. (8). Let $r_{t,i}$ be the ordered role for child position i at level t , τ_t a role for the local node state, and σ_t an optional summary role. The recursive code is

$$H_\nu = \text{Normalize} \left(\tau_t \otimes u_\nu \oplus \sum_{i=1}^{\deg(\nu)} r_{t,i} \otimes \rho(H_{\text{child}_i(\nu)}) \oplus \sigma_t \otimes S_\nu \right). \quad (11)$$

The intuition is that a node’s code is a single superposition of three kinds of tagged content: the node’s own local state, tagged by the state role τ_t ; each child’s recursively encoded subtree, tagged both by which child slot it fills ($r_{t,i}$) and by a permutation ρ that marks depth and prevents sibling paths from colliding; and an optional summary channel for whole-subtree readouts. Because the children enter through distinct role tags, a later query can recover any one of them by unbinding the corresponding role, while the other children act only as bounded crosstalk. Here S_ν is a learned or hand-built summary vector for subtree-level readouts. The normalization `Normalize` specifies whether bundles remain real sums, are projected to bipolar signs, are projected to phasors, or are otherwise quantized. The same choice must be used in the capacity and storage model.

The level codebook used during descent is not implicit. It is the finite set of valid normalized child encodings at that level,

$$\mathcal{K}_t^{\text{sub}} = \{\text{possible } H_\nu \text{ for nodes at level } t\}, \quad (12)$$

or a learned approximation to that set. In practice this can be a VQ dictionary of subtree states, a product-quantized dictionary, a cache of prototypes learned from training latents, or a generated dictionary whose seed and architecture are counted in storage. The theoretical bounds use $M_t^{\text{sub}} = |\mathcal{K}_t^{\text{sub}}|$.

Algorithm 1 Encode a learned factor tree

```

1: procedure ENCODE( $\nu$ , level  $t$ )
2:    $u_\nu \leftarrow \text{Quant}_t(z_\nu)$  ▷ prototype in  $\mathcal{K}_t$ , or product over factor codebooks
3:    $H \leftarrow \tau_t \otimes u_\nu$ 
4:   for  $i = 1$  to  $\text{deg}(\nu)$  do
5:      $C_i \leftarrow \text{ENCODE}(\text{child}_i(\nu), t + 1)$ 
6:      $H \leftarrow H \oplus (r_{t,i} \otimes \rho(C_i))$ 
7:   end for
8:    $S_\nu \leftarrow \text{SUMMARIZE}(\nu)$  ▷ optional task-preserving digest
9:    $H \leftarrow H \oplus (\sigma_t \otimes S_\nu)$ 
10:  return Normalize( $H$ )
11: end procedure

```

Algorithm 2 Descend along a known ordered path

```

1: procedure DESCEND( $H$ , path  $(i_1, \dots, i_\ell)$ )
2:    $h \leftarrow H$ 
3:   for  $t = 0$  to  $\ell - 1$  do
4:      $h \leftarrow \rho^{-1}(r_{t,i_{t+1}}^\dagger \otimes h)$  ▷ target child plus sibling crosstalk
5:      $h \leftarrow \widehat{\text{cleanup}}_{\mathcal{K}_{t+1}^{\text{sub}}}(h)$  ▷ explicit level-codebook cleanup
6:   end for
7:   return  $h$ 
8: end procedure

```

4.2 Path descent

Descent walks down a known path one level at a time. Each step first unbinds the role of the chosen child, which exposes that child’s subtree code superposed with residual crosstalk from its siblings, and then cleans the result against the level’s subtree dictionary to discard that crosstalk and recover a legal subtree code. Without the cleanup on line 5, crosstalk accumulates from off-path subtrees and the effective load can scale with the whole object. With the cleanup and a valid $\mathcal{K}_t^{\text{sub}}$, each step faces only local load.

4.3 Resonator factorization

Intuitively, the resonator factors a product by alternating estimation: it holds all but one factor fixed at their current guesses, unbinds them from H to isolate the evidence bearing on the remaining factor, cleans that evidence up against the corresponding codebook, and cycles through the factors until the hard estimates stop changing. The payoff is that each sweep searches the individual factor codebooks separately, at cost proportional to $\sum_f |\mathcal{C}_f|$, rather than enumerating the full product space $\prod_f |\mathcal{C}_f|$.

The projection step is important: a soft superposition is not generally an invertible atom in MAP or HRR. One can strengthen the practical decoder with multi-start initialization, top- K beams, damping, temperature annealing, and a recombination score

$$\langle H, \hat{x}_1 \otimes \dots \otimes \hat{x}_F \rangle \quad (13)$$

to reject spurious fixed points.

Algorithm 3 Margin-safe resonator factorization of $H \approx x_1 \otimes \cdots \otimes x_F$

```
1: procedure FACTORIZE( $H$ , codebooks  $\mathcal{C}_1, \dots, \mathcal{C}_F$ , iterations  $I$ )
2:   for  $f = 1$  to  $F$  do
3:      $\hat{x}_f \leftarrow \text{Normalize} \left( \sum_{c \in \mathcal{C}_f} c \right)$ 
4:   end for
5:   for  $s = 1$  to  $I$  do
6:     for  $f = 1$  to  $F$  do
7:        $y_g \leftarrow \text{Proj}_{\mathbb{H}}(\hat{x}_g)$  for all  $g \neq f$ 
8:        $z_f \leftarrow H \otimes \left( \bigotimes_{g \neq f} y_g^\dagger \right)$ 
9:        $\hat{x}_f \leftarrow \text{cleanup}_{\mathcal{C}_f}(z_f)$ 
10:    end for
11:    if all hard cleanups are unchanged then
12:      break
13:    end if
14:  end for
15:  return  $(\widehat{\text{cleanup}_{\mathcal{C}_1}(\hat{x}_1)}, \dots, \widehat{\text{cleanup}_{\mathcal{C}_F}(\hat{x}_F)})$ 
16: end procedure
```

4.4 Structured queries

A query can use three access modes.

1. A local path query runs DESCEND, then optionally runs FACTORIZE on the returned node state.
2. A summary query reads a summary channel $\sigma_t \otimes S_\nu$.
3. A homomorphic readout applies a linear or role-equivariant map that has been defined on the encoded representation itself.

The third case needs care. A generic cosine similarity against a content atom at the root is not automatically a count of that content in the underlying tree, because leaf contents are role-bound by their paths. Valid global readouts are linear maps trained directly on root codes, summary-channel readouts explicitly constructed to preserve the statistic, or role-equivariant maps satisfying a known relation such as $\phi(r \otimes x) = A_r \phi(x)$.

5 Theory

The theorems we give here are relatively modest, but. may be of significant practical value nonetheless. They show when the local-load story is valid; they do not claim universal lossless compression. The argument has four steps. We first bound the failure probability of a single cleanup step. We then compose this bound across the few cleanups that a fixed query family actually touches, which gives the central dimension law and shows that the requirement scales with local load and supported queries rather than with raw object size. We then prove a matching lower bound, confirming that some restriction on the supported queries is unavoidable: a code that answered every leaf query would have to pay the full entropy cost. Finally we give a conditional margin theorem for recovering unknown product factors with the resonator.

Algorithm 4 Structured query

```

1: procedure STRUCTUREDQUERY( $H$ , query specification  $q$ )
2:   if  $q$  is a valid root summary or learned linear readout then
3:     return  $\phi_q(H)$ 
4:   end if
5:    $R \leftarrow \emptyset$ 
6:   for each path  $p$  touched by  $q$  do
7:      $h \leftarrow \text{DESCEND}(H, p)$ 
8:     if  $q$  asks for unknown product factors at this node then
9:        $h \leftarrow \text{FACTORIZE}(h, \mathcal{C}_{t,1}, \dots, \mathcal{C}_{t,F_t}, I)$ 
10:    end if
11:     $R \leftarrow R \cup \{\phi_q(h)\}$ 
12:  end for
13:  return AGGREGATE( $\rangle q R$ )
14: end procedure

```

5.1 Cleanup capacity

Definition 2 (Coherence). For a finite set of atoms $\mathcal{A}$, define

$$\mu(\mathcal{A}) = \max_{a \neq b \in \mathcal{A}} \frac{|\langle a, b \rangle|}{D}. \quad (14)$$

Lemma 1 (Deterministic cleanup margin). *Let $z = a_\star + \eta$ where $a_\star \in \mathcal{C}$, $\|a_\star\|^2 = D$, and cleanup is against $\mathcal{C}$. Hard cleanup returns $a_\star$ if*

$$\langle a_\star, z \rangle - \max_{c \in \mathcal{C}, c \neq a_\star} \langle c, z \rangle > 0. \quad (15)$$

In particular, if $\eta = \sum_{j=1}^{k-1} e_j$ and every pairwise normalized overlap among $\mathcal{C} \cup \{e_j\}$ is at most μ , then cleanup succeeds whenever

$$1 > (2k + 1)\mu. \quad (16)$$

Proof sketch. The target score is $D + \langle a_\star, \eta \rangle$ and any distractor score is $\langle c, a_\star \rangle + \langle c, \eta \rangle$. Bounding each off-diagonal inner product by μD gives a positive worst-case margin under Eq. (16). The constant is not optimized; the useful point is that deterministic success is a margin condition. $\square$

Lemma 2 (Random single-step cleanup). *Under Assumptions 1–2, suppose an unbinding step leaves one target atom plus $k - 1$ independent crosstalk terms and cleanup is performed against a codebook of size M . Then for a constant $c > 0$ depending only on the atom distribution,*

$$\mathbb{P}[\text{cleanup fails}] \leq 2M \exp(-cD/k). \quad (17)$$

Proof sketch. The target score has mean D and fluctuations of order $\sqrt{kD}$. Each distractor score is sub-Gaussian with variance proxy $O(kD)$. A union bound over M distractors gives Eq. (17). $\square$

The shape of Eq. (17) is the central scaling law of the paper. Reliability improves exponentially in the ratio D/k of dimension to local load, while the codebook size M enters only linearly inside the bound (logarithmically once one solves for the required D). So the dimension needed to decode reliably grows with the local crosstalk load and only with the logarithm of how many alternatives must be distinguished, not with the size of the whole object. Everything below is essentially this single-step fact composed over the steps a query touches.

Theorem 1 (Fixed-family local query capacity). *Consider a family of encoded factor trees with level subtree codebooks $\mathcal{K}_t^{\text{sub}}$ of sizes M_t , local unbinding loads k_t , and maximum queried depth $\ell_{\max}$. Let $\mathcal{Q}_{\text{sup}}$ be a fixed set of at most $q_{\max}$ path queries. If*

$$D \geq \max_{t \leq \ell_{\max}} \frac{k_t}{c} \log \left(\frac{2q_{\max} \sum_{s=1}^{\ell_{\max}} M_s}{\varepsilon} \right), \quad (18)$$

then all descent cleanups required by all queries in $\mathcal{Q}_{\text{sup}}$ succeed with probability at least $1 - \varepsilon$.

Proof sketch. At level t , Lemma 2 bounds the failure probability by $2M_t \exp(-cD/k_t)$. Union bound over at most $q_{\max}$ queried paths and at most $\ell_{\max}$ levels. Rearranging gives Eq. (18). $\square$

Remark 2 (When dimension is independent of raw size). Equation (18) has no direct N term. However, N can re-enter through $q_{\max}$, $\ell_{\max}$, k_t , or M_t . Thus the dimension is independent of raw object size only in the structured regime where supported queries are limited, depth grows slowly, local branching/load is bounded, and level prototype dictionaries are reusable rather than object-specific archives.

Theorem 2 (All-path lower bound). *Suppose a code supports simultaneous reconstruction of all N leaf labels drawn from an alphabet of size m with error probability bounded below a fixed constant. If each stored component carries at most q_{bits} bits after quantization, then*

$$Dq_{\text{bits}} = \Omega(N \log_2 m). \quad (19)$$

Consequently, no fixed-width code can be lossless for arbitrary N -leaf objects.

Proof sketch. There are m^N possible labelings. A quantized D -component code has at most $2^{Dq_{\text{bits}}}$ states. Fano/counting arguments imply that faithful reconstruction of all labels requires enough states to distinguish an exponential number of labelings, giving the bound. $\square$

5.2 Conditional resonator theorem

Let the target product be

$$y_{\star} = x_{1,\star} \otimes \cdots \otimes x_{F,\star}, \quad x_{f,\star} \in \mathcal{C}_f, \quad (20)$$

and let the observed vector be $H = y_{\star} + \eta$ after any relevant bundling and unbinding.

Definition 3 (Slot update margin). At a resonator iteration, given projected estimates $y_g = \text{Proj}_{\mathbb{H}}(\hat{x}_g)$ for $g \neq f$, define the slot- f evidence

$$z_f = H \otimes \left(\bigotimes_{g \neq f} y_g^{\dagger} \right). \quad (21)$$

The hard-cleanup margin for slot f is

$$\Delta_f = \langle x_{f,\star}, z_f \rangle - \max_{c \in \mathcal{C}_f, c \neq x_{f,\star}} \langle c, z_f \rangle. \quad (22)$$

Theorem 3 (Conditional resonator success). *Assume that during Algorithm 3, every slot update satisfies $\Delta_f \geq \gamma > 0$. Then hard cleanup at each update selects the true factor. For softmax cleanup with inverse temperature β , the probability mass assigned outside the true atom at that update is at most*

$$(|\mathcal{C}_f| - 1) \exp(-\beta\gamma). \quad (23)$$

If the margin condition holds for all slots for I iterations, final hard cleanup returns the true factor tuple.

Proof. The hard statement follows directly from the definition of Δ_f . For the softmax statement, every distractor logit is at least $\beta\gamma$ below the true logit, so the total distractor mass is bounded by a sum of $|\mathcal{C}_f| - 1$ exponentials. Final hard cleanup is correct once every slot’s true atom has the largest score. $\square$

Corollary 1 (Random margin sufficient condition). *If the effective evidence at each slot update is a target atom plus at most $k_f - 1$ independent crosstalk terms and cleanup is against $\mathcal{C}_f$, then the margin condition of Theorem 3 holds for all slots and all iterations with probability at least*

$$1 - 2I \sum_{f=1}^F |\mathcal{C}_f| \exp(-cD/k_f). \quad (24)$$

This is a sufficient condition, not a global convergence theorem from arbitrary initialization.

Remark 3. This form is intentionally weaker than saying resonators always converge. It matches practical use: train or choose codebooks so that correct factors have margin, use projection before inversion, and reject bad fixed points with a recomposition score.

5.3 Task preservation for DNN latents

Let $A_q(z(x))$ denote the answer of query q on the full neural state, and let $A_q(H_x)$ denote the answer returned from the hypervector code. Suppose the query loss is bounded by B_q and is Lipschitz with respect to the factorized latent answer. Let

$$B_{\max} = \sup_{q \in \mathcal{Q}_{\sup}} B_q. \quad (25)$$

Theorem 4 (DNN latent task preservation). *Fix a supported query family $\mathcal{Q}_{\sup}$ of size $q_{\max}$. Assume the DNN factorization defect is $\delta_{\mathcal{Q}_{\sup}}$, level-codebook descent satisfies Theorem 1 with failure probability ε_d , and all resonator calls satisfy Corollary 1 with total failure probability ε_r . Then*

$$\sup_{q \in \mathcal{Q}_{\sup}} \mathbb{E}_x [d_q(A_q(z(x)), A_q(H_x))] \leq \delta_{\mathcal{Q}_{\sup}} + B_{\max}(\varepsilon_d + \varepsilon_r). \quad (26)$$

In particular, the recoverable HDC error can be driven down exponentially in D/k while the task-specific neural factorization defect remains as an irreducible floor.

Proof sketch. Decompose the event into successful local decoding/factorization and failure. On success, the code returns the quantized factor graph answer, whose discrepancy from the full neural state is by definition at most $\delta_{\mathcal{Q}_{\sup}}$. On failure, the loss is bounded by $B_{\max}$. Union bound descent and resonator failures. $\square$

Remark 4 (Causal coding interpretation). If E_θ is trained by a causal or mechanistic objective, the factors in G_x may correspond to stable variables, mechanisms, objects, or interventions. The theorem does not assume that such factors are identifiable from passive raw data alone. It assumes only that a trained or weakly supervised encoder has produced a low-defect factor graph for the chosen query family.

5.4 Storage accounting

Let q_H be the number of bits per stored hypervector component. For bipolar codes $q_H = 1$ after sign quantization. For phasor or real-valued codes q_H is the chosen quantization precision. For a dataset of n items,

$$B_{\text{code}} = nDq_H + B_{\text{roles}} + B_{\text{codebooks}} + B_{\text{models}}. \quad (27)$$

If roles and codebooks are pseudorandomly generated from seeds, count the seeds and generator descriptions. If subtree prototypes or DNN quantizers are stored explicitly, count them. If the DNN encoder is required at query time, count its model storage and compute separately from the cached code. The per-item rate is small only when shared dictionaries and models are amortized over many items.

5.5 Speed and baseline accounting

Let $C_{\text{HDC}}(q)$ be the cost of running query q on the code. A local query touching p paths of depth at most ℓ has cost

$$C_{\text{HDC}}(q) = O\left(p\ell D \max_t M_t + ID \sum_f |\mathcal{C}_f|\right) \quad (28)$$

when naive dense cleanup is used. Approximate nearest-neighbor cleanup, structured codebooks, binary operations, or hardware parallelism can reduce constants.

Proposition 1 (Speed baselines). *The meaningful speedup depends on the baseline.*

1. *Against an unindexed raw or latent scan costing $\Theta(N)$, a root summary or learned linear readout costing $O(D)$ has scan speedup*

$$\Sigma_{\text{scan}} = \Theta(N/D). \quad (29)$$

2. *Against a standard indexed tree path lookup costing $O(\ell)$, a local HDC path query costing Eq. (28) is not asymptotically faster. Its possible advantages are compression, batched computation, noise robustness, learned summaries, or hardware efficiency.*

3. *Against brute-force factor search over $\prod_f |\mathcal{C}_f|$ tuples, the resonator update cost $O(ID \sum_f |\mathcal{C}_f|)$ is exponential-to-linear in the factor codebook sizes, subject to the margin condition.*

5.6 Compression-compute accounting

For a scan baseline and a single local query of fixed depth ℓ , define an amortized raw latent size of $N \log_2 m$ bits and an HDC item size of Dq_H bits. The compression ratio is

$$\rho = \frac{Dq_H}{N \log_2 m}. \quad (30)$$

Using a local HDC cost proportional to $D\ell C_{\text{step}}$, the scan speedup is

$$\Sigma_{\text{loc,scan}} = \Theta\left(\frac{N}{D\ell C_{\text{step}}}\right), \quad (31)$$

and therefore

$$\rho \Sigma_{\text{loc,scan}} = \Theta\left(\frac{q_H}{\log_2 m \ell C_{\text{step}}}\right). \quad (32)$$

For fixed depth, the product is independent of D and N ; if $\ell = \log_b N$, it carries the expected logarithmic dependence on N . Shrinking D improves both nominal storage and HDC query cost, but only down to the fidelity threshold in Eq. (18). The real tradeoff is not compression versus speed; it is compression and speed versus distortion.

6 Scalability objections as design criteria

The theory above is useful only if its assumptions can be checked. The scope of potential objections to hierarchical HDC, some of which we review here, are therefore best treated not as side issues but as measurable design criteria.

6.1 Effective rather than enumerative subtree dictionaries

A literal dictionary of every possible subtree at level t can be exponentially large. The theory should therefore not be interpreted as requiring enumeration of all generative compositions. Instead, the relevant quantity is an effective cleanup dictionary for the states that the task family actually distinguishes. Let

$$M_t^{\text{eff}}(\gamma) = N(\mathcal{Z}_t, d_{\text{HDC}}, \gamma) \quad (33)$$

be the covering number of observed or task-relevant level- t latent states at cleanup margin scale γ , where d_{HDC} is the similarity metric used by cleanup. The local dimension condition can then be read as

$$D \gtrsim \max_t k_t \left[\log M_t^{\text{eff}}(\gamma) + \log(q_{\max} \ell_{\max} / \varepsilon) \right]. \quad (34)$$

The method scales when the upstream learner collapses many raw configurations into a moderate number of reusable, well-separated latent states. It should fail when every subtree is essentially unique for the task.

For highly compositional nodes, a better strategy is factored cleanup rather than monolithic subtree cleanup:

$$u_\nu = c_{\nu,1} \otimes \cdots \otimes c_{\nu,F_t}, \quad c_{\nu,f} \in \mathcal{C}_{t,f}. \quad (35)$$

The resonator then searches over local factor dictionaries rather than over the explicit product dictionary. The practical advantage is not that the product space disappears, but that a high-margin product can be recovered by iterated slot-wise cleanup with cost proportional to $\sum_f |\mathcal{C}_{t,f}|$ rather than $\prod_f |\mathcal{C}_{t,f}|$.

6.2 Binding must be fixed or quasi-isometric

The proof of local cleanup assumes that binding and unbinding do not amplify crosstalk. This is reliable when binding is a fixed VSA operation such as Hadamard multiplication, XOR, phasor multiplication, permutation, or an orthogonal/unitary block map. If binding is learned freely by backpropagation, the system should not be expected to remain isometric.

A learnable binding family B_a should instead be constrained by a quasi-isometry condition on the relevant latent set:

$$|\langle B_a u, B_a v \rangle - \langle u, v \rangle| \leq \eta_{\text{bind}} D. \quad (36)$$

Then every cleanup theorem should be read with an effective margin

$$\gamma_{\text{eff}} = \gamma - C_{\text{path}} \eta_{\text{bind}} D - C_{\text{scale}} \eta_{\text{scale}} D, \quad (37)$$

where C_{path} depends on the number of binding/unbinding operations in the query. In practical systems, the safest design is to let the DNN learn the factors and codebooks while keeping the HDC binding algebra fixed or strongly constrained.

6.3 Resonator search cost must be charged

A conditional margin theorem is honest, but it does not by itself guarantee that retrieval is cheap. If practical decoding uses R restarts, beam width K , I iterations, and dense cleanup over factor codebooks, then the local product search cost is

$$C_{\text{res}} = O \left(RKID \sum_{f=1}^F |\mathcal{C}_f| \right), \quad (38)$$

plus any recomposition scoring. The resonator is computationally useful only when this is smaller than the avoided baseline: raw scan, dense attention over memory, full DNN recomputation, or brute-force enumeration of factor tuples. Approximate nearest-neighbor cleanup, structured codebooks, binary HDC hardware, and good warm starts can reduce constants, but they should be reported rather than hidden.

6.4 The factorization defect is a test, not a rescue clause

The task-specific defect $\delta_{\mathcal{Q}}$ absorbs quantization error, missing nonlocal interactions, and holistic synergies. It should not be used to make the theorem vacuously true. It is the criterion that determines whether HDC is relevant to a task. The empirical protocol should always report

$$\Delta_{\text{fact}} = \mathcal{L}_{\text{fact}} - \mathcal{L}_{\text{full}}, \quad (39)$$

$$\Delta_{\text{HDC}} = \mathcal{L}_{\text{HDC}} - \mathcal{L}_{\text{fact}}. \quad (40)$$

If Δ_{fact} is large, the upstream factorization is inadequate. If Δ_{fact} is small but Δ_{HDC} is large, the HDC code is under-dimensioned, overloaded, geometrically distorted, or decoded with an insufficient search budget.

6.5 Admissibility checklist

For a target query family $\mathcal{Q}$, a learned or hand-built factorization should be called HDC-admissible at dimension D only if the following quantities are small or favorable:

$$\boxed{\delta_{\mathcal{Q}}, \quad k_t, \quad \log M_t^{\text{eff}}, \quad M_{\text{conf}}, \quad RKI, \quad \eta_{\text{bind}}} \quad (41)$$

and the cleanup margin exceeds the predicted crosstalk and binding distortion. This checklist is also the bridge to learning: these are quantities the upstream system can be trained to improve.

7 HDC-guided factorization learning

The HDC layer is most plausible as a cache over learned or constructed factors, not as a direct replacement for a perception, language, or planning model. Its novel role is stronger than compression: it gives the upstream learner an operational target. The learner should not merely find a small latent. It should find a latent factor graph that is easy to bind, bundle, clean up, resonate, and query.

7.1 Representation families that can supply factors

Several existing neural representation families naturally supply pieces of the model.

Vector quantized latents. VQ encoders produce discrete latent codes. A multiscale VQ model supplies level codebooks $\mathcal{K}_t$ and reusable prototypes. The HDC code then bundles, binds, and queries these prototypes rather than storing every raw input coordinate.

Object-centric and slot latents. Slot or object-centric encoders produce sets of latent object states. HDC roles can bind object identity, position, time, relation, occlusion status, and scale. The local load k_t becomes the number of objects or relations that compete at a given scale.

Sparse feature dictionaries for language. Sparse autoencoders over language-model activations provide overcomplete feature dictionaries. A document, sentence, or activation window can be represented as a sparse set of active features bound to token position, syntactic role, discourse role, layer, or head. The defect is low only for probes preserved by the dictionary; it is not a guarantee for full next-token generation.

Causal or mechanistic latent codes. A causal-coding encoder aims to discover variables corresponding to stable mechanisms, interventions, object states, or transition factors. If such a code has bounded local parents and reusable state prototypes, it directly satisfies the local-load assumptions. If the variables are not identifiable or the task depends on nonlocal synergies, this appears as a large $\delta_{\mathcal{Q}}$ rather than as an HDC dimension problem.

Structured non-neural algorithms. The same idea applies beyond DNNs. A parser, theorem prover, program analyzer, database engine, causal-discovery procedure, planner, or symbolic reasoning system may already maintain a structured internal state. If that state has roles, local operations, reusable prototypes, or bounded-width factorization, the HDC layer can shape and cache it in the same way.

7.2 HDC-admissible factorization

The upstream learner should seek query-factorized sufficiency rather than global disentanglement. For a query family $\mathcal{Q}$, the factor graph G_x is useful when

$$q(E_{\theta}(x)) \approx q(G_x), \quad q \in \mathcal{Q}, \quad (42)$$

while each query touches only a few role-bound factors. A concise capacity proxy is

$$\mathcal{C}_{\text{HDC}} = \max_t k_t^{\text{eff}} \left[\log M_t^{\text{conf}} + \log(q_{\max} \ell_{\max} / \varepsilon) \right]. \quad (43)$$

The dimension requirement is roughly $D \gtrsim c^{-1} \mathcal{C}_{\text{HDC}}$. The HDC layer therefore asks the learner to reduce local load, reduce confusable alternatives, and increase cleanup margins while preserving task sufficiency.

7.3 Measuring factorization error during learning

The basic diagnostic uses three readouts:

$$z(x) \rightarrow \hat{y}_{\text{full}}, \quad (44)$$

$$G_x \rightarrow \hat{y}_{\text{fact}}, \quad (45)$$

$$H_x \rightarrow \hat{y}_{\text{HDC}}. \quad (46)$$

Then

$$\mathcal{L}_{\text{HDC}} - \mathcal{L}_{\text{full}} = \Delta_{\text{fact}} + \Delta_{\text{HDC}} \quad (47)$$

with

$$\Delta_{\text{fact}} = \mathcal{L}_{\text{fact}} - \mathcal{L}_{\text{full}}, \quad \Delta_{\text{HDC}} = \mathcal{L}_{\text{HDC}} - \mathcal{L}_{\text{fact}}. \quad (48)$$

This separates representation failure from HDC coding failure.

7.4 Local load loss

For a query q and example x , let $a_i(q, x)$ be the relevance or attention weight assigned to factor i . The effective number of factors touched by the query is the participation ratio

$$k_{\text{eff}}(q, x) = \frac{1}{\sum_i a_i(q, x)^2}. \quad (49)$$

A load regularizer is

$$\mathcal{L}_{\text{load}} = \mathbb{E}_{x,q} [\max(0, k_{\text{eff}}(q, x) - k_{\text{budget}})^2]. \quad (50)$$

This encourages the learner to organize information so that supported queries are local in the HDC sense.

7.5 Cleanup margin and confusability losses

For a latent vector z_ν assigned to prototype c^+ , define

$$\Delta(z_\nu) = \langle z_\nu, c^+ \rangle - \max_{c^- \neq c^+} \langle z_\nu, c^- \rangle. \quad (51)$$

The margin loss is

$$\mathcal{L}_{\text{margin}} = \left[\gamma + \max_{c^- \neq c^+} \langle z_\nu, c^- \rangle - \langle z_\nu, c^+ \rangle \right]_+. \quad (52)$$

The margin target should scale with the predicted crosstalk. In normalized cosine form, a useful heuristic is

$$\gamma_t^{\cos} \approx C \sqrt{\frac{k_t \log M_t}{D}}. \quad (53)$$

The number of dangerous competitors is

$$M_{\text{conf}}(z_\nu) = \# \{c : \langle z_\nu, c \rangle \geq \langle z_\nu, c^+ \rangle - \gamma\}, \quad (54)$$

and can be penalized by $\mathbb{E} \log M_{\text{conf}}(z_\nu)$. This is a better quantity than nominal codebook size when only a few states are actually confusable.

7.6 Product recombination and resonator identifiability

If a node latent is intended to factor as

$$z_\nu \approx c_{\nu,1} \otimes \cdots \otimes c_{\nu,F}, \quad (55)$$

then train not only for reconstruction but also for recoverability:

$$\mathcal{L}_{\text{prod}} = \|z_\nu - \text{Normalize}(c_{\nu,1} \otimes \cdots \otimes c_{\nu,F})\|^2, \quad (56)$$

while periodically measuring

$$p_{\text{res}} = \mathbb{P}[\text{FACTORIZE}(z_\nu) \neq (c_{\nu,1}, \dots, c_{\nu,F})]. \quad (57)$$

A product code with low reconstruction error but high resonator failure rate is not HDC-admissible.

7.7 Role stability

A reusable content factor should remain stable when it appears in different roles. If $z_a \approx r_a \otimes c$ and $z_b \approx r_b \otimes c$, then

$$r_a^\dagger \otimes z_a \approx r_b^\dagger \otimes z_b. \quad (58)$$

The role-consistency loss on paired or augmented examples is

$$\mathcal{L}_{\text{role}} = \left\| r_a^\dagger \otimes z_a - r_b^\dagger \otimes z_b \right\|^2. \quad (59)$$

For video this is object persistence; for language it is entity or predicate stability across syntactic and discourse roles; for causal coding it is mechanism stability across contexts.

7.8 Residual synergy test

Let $h_{\text{full}} = E_\theta(x)$ and let $h_{\text{fact}} = G_\omega(F_\psi(E_\theta(x)))$ be a reconstruction of the full latent from the factor graph. The residual

$$r = h_{\text{full}} - h_{\text{fact}} \quad (60)$$

should not contain task-relevant information. A residual probe $r \rightarrow \hat{y}_r$ estimates whether the factorization missed important synergy. If the residual probe performs well conditional on G_x , then δ_Q is high.

7.9 Intervention locality for causal coding

When paired examples differ by an intervention on factor j , the desired pattern is

$$c_i(x) \approx c_i(x') \quad (i \neq j), \quad c_j(x) \not\approx c_j(x'). \quad (61)$$

A simple intervention locality loss is

$$\mathcal{L}_{\text{intervention}} = \sum_{i \neq j} d(c_i(x), c_i(x')) - \lambda d(c_j(x), c_j(x')). \quad (62)$$

This is exactly aligned with the HDC requirement that interventions change only a bounded number of role-bound factors.

7.10 Total HDC-guided objective

A representative training objective is

$$\mathcal{L} = \mathcal{L}_{\text{task}}^{\text{HDC}} + \lambda_{\delta} \Delta_{\text{fact}} + \lambda_{\text{margin}} \mathcal{L}_{\text{margin}} + \lambda_{\text{load}} \mathcal{L}_{\text{load}} \quad (63)$$

$$+ \lambda_{\text{prod}} \mathcal{L}_{\text{prod}} + \lambda_{\text{role}} \mathcal{L}_{\text{role}} + \lambda_{\text{cap}} \mathcal{L}_{\text{HDC}} + \lambda_{\text{res}} \mathcal{L}_{\text{residual}}. \quad (64)$$

The point is not that every application must use every term. The point is that HDC supplies measurable pressure toward a particular kind of factorization: reusable discrete or nearly discrete factors such that each supported query depends on a small number of well-separated role-bound factors, with little task-relevant information left in the residual.

7.11 Live diagnostics

During training one should track

$$\mathcal{L}_{\text{full}}, \quad \mathcal{L}_{\text{fact}}, \quad \mathcal{L}_{\text{HDC}}, \quad k_{\text{eff}}, \quad M^{\text{eff}}, \quad M_{\text{conf}}, \quad \Delta, \quad (65)$$

plus resonator success rate, resonator iterations, role-consistency error, and residual-probe accuracy. The learning process is moving in the HDC-admissible direction when

$$\mathcal{L}_{\text{fact}} \approx \mathcal{L}_{\text{full}}, \quad \mathcal{L}_{\text{HDC}} \approx \mathcal{L}_{\text{fact}}, \quad (66)$$

while $k_{\text{eff}} \log M_{\text{conf}}$ decreases, cleanup margins increase, and residual-probe accuracy falls.

8 Resonant modular compositional episodic memory and world modeling

We now spell out a second major application of the general theory: episodic memory and world modeling in an architecture that uses modular compositional hypervectors as an intermediate representational regime. The key observation is simple but, in our view, important. If an episode itself has a factorized structure, then the episode should not be collapsed into a monolithic dense embedding before being stored. It should be compiled into a typed, role-filler, modular hypervector structure, and the resonator machinery should be allowed to operate on that structure.

This yields what we will call resonant hierarchical modular hypervectors, or R-HMH. The HMH side supplies the typed, ontology-aligned, named-slot address space; the resonator side supplies factor recovery, cleanup, and a quantitative pressure toward locally decodable low-load factors. The resulting memory item is not just a vector similar to an old experience. It is a compact cache of a structured episode whose parts can be separately queried, completed, swapped, aggregated, and consumed by neural or symbolic downstream systems.

8.1 Review: HMH and PRIMUS-style world modeling

Hierarchical Modular Hypervectors combine two ideas. First, type-based hierarchical named embeddings assign semantically meaningful slots to ontology types: for instance Demand Subject, Demand Object, Effect, Precondition, Location, Time, Value, Social Role, or Evidence. Second, modular compositional hypervectors use fixed-length hypervectors, role-filler binding, bundling, permutation, unbinding, cleanup, and approximate nearest-neighbor retrieval. The synthesis is an embedding whose slices or modules have human-interpretable functional meaning, while the

whole representation remains fixed-dimensional, noise-tolerant, algebraically compositional, and queryable by inverse binding [13, 14].

In a PRIMUS-style world model, HMM is not merely a compact storage format for episodes. It is a third representational regime between explicit symbolic atoms and dense neural tensors [15]. Symbolic atoms and rules carry crisp or uncertain declarative structure; dense vectors and tensors support perceptual prediction, control, and gradient learning; HMM stores structured associative keys over episodes, skills, affordances, social scripts, options, and mined templates. A map such as

$$\mathcal{D}_{\text{hmm}} : \mathbb{H}_D \rightarrow \mathbb{R}^d \quad (67)$$

then produces dense vectors usable by neural modules, while preserving, as much as possible, both the similarity geometry and the algebraic role-filler structure compiled into the HMM code.

This is exactly the setting in which resonator-HDC should be useful. A world model must often answer questions such as:

Which prior interaction had the same social role pattern but a different actor? Which affordance trial had the same causal preconditions but a different object? Which episode had the same affective trajectory and repair outcome? Which skill was invoked in contexts with this goal, tool, and failure mode?

These are not whole-episode reconstruction queries. They are local, role-conditioned, factor-aware retrieval and completion queries.

8.2 A factorized episode model

Let an episode E be represented by a small typed factor graph

$$G_E = (S_E, C_E, A_E), \quad (68)$$

where S_E is a set of slot factors, C_E is a set of typed relations among slot factors, and A_E is a set of anchors to symbolic atoms, perceptual shards, logs, media, or motor traces. Typical slots include

$S_E = \{\text{actor, object, recipient, action, place, time, goal, plan, norm,}$
 $\text{affect, belief, tool, affordance, precondition, cause, outcome,}$
 $\text{reward, uncertainty, skill, script, evidence, perceptual trace}\}.$

The relation set C_E may include temporal order, causation, enablement, spatial adjacency, ownership, social obligation, repair, conflict, co-reference, part-whole structure, and counterfactual alternatives.

A basic R-HMM episode code has the form

$$H_E = \text{Normalize} \left(r_{\text{schema}} \otimes h_{\sigma(E)} \oplus \sum_{s \in S_E} m_{\tau(s)} \otimes r_s \otimes h_{f_s} \right. \\ \left. \oplus \sum_{(i,j,\rho) \in C_E} r_\rho \otimes r_i \otimes r_j \otimes (h_{f_i} \otimes h_{f_j}) \oplus \sum_{a \in A_E} r_{\text{anchor}(a)} \otimes h_a \right). \quad (69)$$

Here $h_{\sigma(E)}$ is a schema or event-type code, $m_{\tau(s)}$ is a module marker for the ontology type of slot s , r_s is the role vector for that slot, h_{f_s} is the filler code, r_ρ is a relation-role vector, and h_a is an anchor code pointing to external evidence or symbolic structure. The same item therefore has

three linked aspects: a symbolic graph, an HMM key, and optional perceptual or motor payloads. The HMM key does not replace the symbolic graph or the evidence store; it makes the episode addressable and partially decodable by vector-symbolic operations.

For example, a social interaction episode might bind teacher, child, interruption, embarrassment, repair, classroom, and later-positive-outcome into their named slots, while also binding relation factors such as causes(interruption, embarrassment) and repairs(apology, embarrassment). A Minecraft affordance trial might bind candidate tool, construction goal, adjacency pattern, redstone-like state, trial action, observed effect, and uncertainty into the same general format.

8.3 Resonant partial recall and slot completion

Given a cue Q , retrieval proceeds in two stages. An approximate-nearest-neighbor index first returns candidate episode codes by coarse similarity. Then role-masked unbinding and resonator cleanup recover or complete the factors relevant to the query. A cue such as

$$HQ = r_{\text{social role}} \otimes h_{\text{teacher}} \oplus r_{\text{affect}} \otimes h_{\text{embarrassment}} \oplus r_{\text{outcome}} \otimes h_{\text{repair}} \quad (70)$$

should retrieve episodes sharing the social-affective-causal pattern even when specific names, utterances, or objects differ. Conversely, a cue such as

$$HQ = r_{\text{affordance context}} \otimes h_{\text{wet_adjacent_circuit}} \oplus r_{\text{goal}} \otimes h_{\text{open_door}} \quad (71)$$

should retrieve tool-use or construction episodes with structurally similar causal affordance patterns.

When a retrieved or queried factor is itself a product, resonator cleanup can be used inside the episode. Suppose a slot filler is intended to factor as

$$h_{f_s} \approx c_{s,1} \otimes c_{s,2} \otimes \cdots \otimes c_{s,F_s}, \quad c_{s,j} \in \mathcal{C}_{s,j}. \quad (72)$$

For a partial cue, the decoder iteratively forms evidence for one factor at a time,

$$z_{s,j} = \tilde{h}_{f_s} \otimes \left(\bigotimes_{g \neq j} \text{Proj}_{\mathbb{H}}(\hat{c}_{s,g})^\dagger \right), \quad \hat{c}_{s,j} \leftarrow \text{cleanup}_{\mathcal{C}_{s,j}}(z_{s,j}), \quad (73)$$

where $\tilde{h}_{f_s}$ is the slot estimate obtained by unbinding the appropriate role and module markers from H_E or from a retrieved candidate. This gives a natural mechanism for recovering unknown factors such as the likely goal, social role, affordance state, or missing outcome, provided the same margin conditions from Section 5 are satisfied.

The advantage is not that the memory can hallucinate arbitrary missing details. The advantage is that it can complete a bounded family of structured questions by searching factor codebooks rather than entire episodes. It can ask, for example, “which outcome factor makes this remembered affordance trial coherent with this goal and tool?” or “which social-script factor best completes this interaction pattern?”

8.4 Factor-aware neural judgment

A central motivation for PRIMUS-style use of HMM is that neural systems should be able to consume structured memories without losing the structure that made them useful. Let

$$z_E = \mathcal{D}_{\text{hmm}}(H_E) \quad (74)$$

be a dense representation of the episode. In the intended use, z_E is “dual-channel”: one part preserves similarity geometry for neural matching, while another part preserves enough algebraic information to support role-filler sensitivity. A judgment model may then have the form

$$J_\phi(E, Q) = \Phi_\phi(\mathcal{D}_{\text{hnh}}(H_E), \mathcal{D}_{\text{hnh}}(H_Q), m_{\text{slot}}, m_{\text{causal}}, m_{\text{temporal}}), \quad (75)$$

where the masks identify which slots, relations, or temporal windows are relevant to the current judgment. This lets a neural policy, critic, dialogue module, or world-model predictor condition on retrieved episodes while retaining access to who, what, where, why, with what causal role, and with what evidential support.

Counterfactual factor swaps are especially useful as training data. In a real-sum hypervector implementation one can form, schematically,

$$H_E[s := f'] = H_E - m_{\tau(s)} \otimes r_s \otimes h_{f_s} + m_{\tau(s)} \otimes r_s \otimes h_{f'}. \quad (76)$$

In sign or phasor systems the same operation is implemented by rebuilding the slot bundle from its factor record rather than by literal vector subtraction. The training question is then whether the judgment should be invariant or sensitive to the swap. Changing actor identity may be irrelevant for some affordance predictions but crucial for a social-trust prediction; changing causal relation or outcome should often matter. Thus the memory representation directly supplies counterfactual supervision that respects the episode’s factorization.

8.5 Episodic-semantic consolidation

R-HMH also gives a concrete mechanism for episodic-semantic braiding. Repeated retrieval of related episodes exposes common factor patterns:

$$\{H_{E_1}, \dots, H_{E_n}\} \longrightarrow H_{\text{template}} = \text{Normalize} \left(\sum_{i=1}^n w_i H_{E_i}^{\text{aligned}} \right), \quad (77)$$

where the alignments unbind and rebind corresponding slots so that different actors, objects, or contexts can be abstracted into role-level templates. A cluster of tool-use trials can become an affordance schema; a cluster of interaction repairs can become a social script; a cluster of repeated action sequences can become a skill or option template. The resulting template can be stored back as a symbolic rule, as an HMH associative key, and as a densified neural context vector.

The same process supplies a useful memory-management criterion. An episode should be retained when it has high future retrieval value, high evidence value, or high template-forming value. It should be consolidated when its idiosyncratic perceptual details are no longer needed for most supported queries but its factor pattern recurs. It should be re-encoded or pruned when ontology changes, cleanup margins fall, or residual probes show that important task information is no longer captured by the stored factors.

8.6 Capacity criteria for episodic memory

The capacity law from Section 5 now becomes an episodic-memory criterion. For a supported family of episodic queries $\mathcal{Q}_{\text{epis}}$, define an episodic factorization defect

$$\delta_{\mathcal{Q}_{\text{epis}}}^{\text{epis}} = \sup_{q \in \mathcal{Q}_{\text{epis}}} \mathbb{E}_E [d_q(q(\text{full episode state}), q(G_E))]. \quad (78)$$

The R-HMH memory is useful only if this defect is small for the questions the agent actually asks. Its HDC coding error is then governed by effective slot load, relation load, confusable codebook size, cleanup margin, query depth, and the number of supported episodic queries.

A practical admissibility checklist is:

$$\boxed{k_{\text{slot}}, \quad k_{\text{rel}}, \quad \log M_{\text{conf}}, \quad \Delta_{\text{cleanup}}, \quad \delta_{\mathcal{Q}_{\text{epis}}}^{\text{epis}}, \quad RKI, \quad \eta_{\text{bind}}} \quad (79)$$

where k_{slot} is the number of simultaneously active slot factors, k_{rel} is the load due to typed relations among them, M_{conf} is the number of dangerous alternatives in the relevant cleanup dictionaries, Δ_{cleanup} is the measured cleanup margin, RKI is the resonator search budget, and η_{bind} is binding distortion. If every episode requires a large holistic context vector to answer the task’s questions, then the defect term is large and R-HMH is not the right representation by itself. If the factor graph preserves the supported questions but retrieval fails, then the HDC code is under-dimensioned, overloaded, or decoded with insufficient search.

8.7 Operational workflow

A complete R-HMH episodic-memory loop can be summarized as follows.

1. **Ground and segment.** Perception, symbolic grounding, and local world-model inference create an episode graph G_E with slots, typed relations, and evidence anchors.
2. **Encode.** The graph is compiled into H_E by Eq. (69), while raw percepts and motor traces remain in specialized stores.
3. **Retrieve.** A symbolic or dense context forms an HMH query. AtomSpace or another symbolic store may pre-filter candidates; ANN search retrieves nearby HMH items.
4. **Decode and complete.** Role unbinding, module masks, and resonator cleanup recover the slots or product factors needed by the current query.
5. **Densify and judge.** Retrieved items are mapped through $\mathcal{D}_{\text{hmm}}$ and used by neural policies, predictors, critics, or dialogue modules, with masks that preserve factor relevance.
6. **Consolidate.** Repeated high-value retrievals are abstracted into schemas, scripts, affordance templates, skills, or options, which are stored back in symbolic, HMH, and dense forms.

The result is a memory layer whose unit of storage is not a flat remembered scene, but a resonantly decodable factor graph cache. It can support partial recall, analogical retrieval, counterfactual judgment, evidence-grounded replay, and schema formation, while remaining compatible with the query-limited capacity theory developed above.

9 Instantiation in HBCML and ColBaC

The preceding sections are deliberately architecture-agnostic. They apply to any neural or algorithmic system that exposes reusable role-bound factors. The fit is particularly direct for Hierarchical Bayesian Causal Modular Learning (HBCML) and Columnar Bayesian Causal Coding (ColBaC), because these architectures are already designed around sparse causal modules, internal certificates, local teachers, and reusable structure.

9.1 HBCML background

An HBCML system is a tuple

$$\mathcal{H} = (\mathcal{N}, C, \Phi, \Psi, T). \quad (80)$$

Here $\mathcal{N} = \{N_1, \dots, N_M\}$ is a family of structurally restricted component learners, C combines the outputs of the active components, Φ is the family of internal probabilistic states of the components, Ψ is a top-level probabilistic controller that selects supports and decides reuse, diversification, or recruitment, and T is a family of local counterfactual teachers. For context t , the controller selects a sparse support $S_t \subset \{1, \dots, M\}$ and the output has the schematic form

$$\hat{y}_t = C((N_m(x_t; \theta_m, \phi_m))_{m \in S_t}; \Psi). \quad (81)$$

The HBCML design decomposes uncertainty across two structural scales. The top level faces a sparse combinatorial selection problem: which components should be active in this context? Inside each component, a second probabilistic process tracks whether internal material is reusable causal structure, semi-general stabilizing structure, task-local residue, or stale material due for recycling. The inside of a component emits certificates upward, and the controller uses those certificates to improve future selection.

The architecture is motivated by continual learning. If each task has an intended sparse support, gradients outside that support are small, cross-Hessian blocks between support and complement are small, and the combiner is Lipschitz, then the induced task update fields have small commutators. Sequential learning then approximates joint learning up to a controlled modularity error. HBCML is a constructive architecture designed to make these conditions hold by design, rather than hoping that a generic dense network discovers them on its own.

9.2 ColBaC background

ColBaC is a concrete columnar instance of HBCML. A ColBaC model contains a pool of columns. Each example activates a sparse subset of columns, plus any always-on shared columns. Each column contains typed microcolumns, typically

$$K \quad \text{kernel-center processing}, \quad L \quad \text{lateral local refinement}, \quad B \quad \text{bridge or cross-scale context}. \quad (82)$$

Each microcolumn contains a protected hard kernel and adaptable shell tiers. The hard kernel is non-prunable and carries stable structure. Inner shells carry reusable abstraction, middle shells carry semi-general stabilizing material, and outer shells carry task-local exploratory residue. Promotion from outer to inner shells reflects consolidation; demotion outward reflects controlled forgetting.

The top-level controller can be trained in a teacher-first regime. At small scale, exact or exhaustive support search is possible. During learning, a local one-swap teacher can evaluate supports differing by one column and apply a swap only if the audited objective improves. A second teacher can audit internal shell swaps. This provides conservative local supervision for later heuristic or learned support-selection policies.

Internal certificates are central. A column can report shared-abstraction mass, specificity load, saturation pressure, demotion pressure, similarity signatures, probe influence, or other estimates of reuse utility. In Bayesian terms, conditioning the support controller on informative certificates can only improve the optimal mean-squared estimate of latent reuse utility. In engineering terms, the certificates tell the controller which columns are reusable, overloaded, stale, or close to the current context.

9.3 Why HDC fits ColBaC

ColBaC supplies the factorization that HDC needs: sparse supports, typed microcolumns, shell tiers, reusable motifs, and local teachers. HDC supplies what ColBaC needs: an algebraic representation of column identity, motif content, shell hygiene, support context, and certificates. The natural synthesis is

ColBaC learns causal columns; HDC makes their reusable contents addressable, compositional, and auditable. (83)

For column m , define a column hypervector

$$H_m(x) = \text{Normalize} \left(\sum_{u \in \{K, L, B\}} \sum_{s \in \{0, 1, 2, 3\}} \sum_{a \in A_m(x)} r_u \otimes r_s \otimes r_a \otimes c_{m, u, s, a}(x) \right). \quad (84)$$

Here u is the microcolumn type, $s = 0$ may denote the hard kernel and $s = 1, 2, 3$ the shells, a is a patch/time/context role, and $c_{m, u, s, a}$ is a cleaned-up motif atom or factored local product. In words, a column’s code is the superposition of all its active motifs, with each motif triple-tagged by its microcolumn type, its shell tier, and its local role; this is exactly the hierarchical encoder of Eq. (11) specialized to the column’s internal structure, so the same unbinding-and-cleanup machinery can read any tagged piece back out. The active support code is

$$H_{S_t}(x) = \text{Normalize} \left(\sum_{m \in S_t} r_m \otimes H_m(x) \right). \quad (85)$$

This code can be used for support retrieval, context memory, certificate matching, and auditing.

9.4 HDC certificates

A scalar certificate vector can be replaced or augmented by a structured certificate hypervector

$$Z_m = r_{\text{shared}} \otimes z_{\text{shared}, m} \oplus r_{\text{specific}} \otimes z_{\text{specific}, m} \oplus r_{\text{saturation}} \otimes z_{\text{saturation}, m} \quad (86)$$

$$\oplus r_{\text{demotion}} \otimes z_{\text{demotion}, m} \oplus r_{\text{signature}} \otimes H_m. \quad (87)$$

The controller can then query structured certificate content by unbinding roles or by similarity against a context query vector. For example, it can ask for columns with high shared abstraction, low saturation, and a signature close to the current context.

A simple HDC support score is

$$\text{score}(m \mid x) = \langle Q(x), P_m \rangle + \lambda_Z \langle Q_Z(x), Z_m \rangle - \lambda_{\text{sat}} \text{saturation}(m) - \lambda_{\text{churn}} \mathbf{1}[m \notin S_{t-1}], \quad (88)$$

where $Q(x)$ is a context hypervector, P_m is the learned column prototype, and Z_m is the current certificate. The one-swap teacher audits the top retrieved supports and updates the prototypes and score model from the audit outcome.

9.5 HDC-guided shell hygiene

The HDC layer gives a precise rule for promotion and demotion. A motif should move inward when it is reused across contexts, has high cleanup margin, is stable under role unbinding, improves task loss through the factorized/HDC readout, and is not confusable with many other motifs. A motif

should move outward or be recycled when reuse falls, margin falls, confusability rises, specificity load rises, or a residual probe shows that it no longer captures the intended task-relevant structure.

Thus shell dynamics can use HDC quantities:

$$\text{promote}(c) \Leftarrow \text{reuse}(c) \uparrow, \quad \Delta(c) \uparrow, \quad M_{\text{conf}}(c) \downarrow, \quad \Delta_{\text{fact}} \downarrow, \quad (89)$$

$$\text{demote}(c) \Leftarrow \text{reuse}(c) \downarrow, \quad \Delta(c) \downarrow, \quad M_{\text{conf}}(c) \uparrow, \quad \text{specificity}(c) \uparrow. \quad (90)$$

The hard kernel should use fixed or nearly fixed HDC atoms. Inner shells should use high-margin cleanup. Middle shells can use soft cleanup or mixture codes. Outer shells should receive weak HDC regularization at first, to avoid premature discretization of exploratory material.

9.6 The bridge microcolumn

The B microcolumn is both powerful and dangerous. It carries cross-region, cross-scale, or cross-time context. In HDC terms, it is the channel for nonlocal bindings. If B becomes dense global attention, the local-load theorem no longer applies. The design rule is to force bridge content through sparse typed relations:

$$H_{B,m} = \sum_{j=1}^{k_B} r_{\text{rel},j} \otimes r_{\text{src},j} \otimes r_{\text{tgt},j} \otimes c_{\text{rel},j}. \quad (91)$$

The bridge budget k_B should be measured and regularized just like any other local load.

9.7 Episodic support memory

A ColBaC-HDC memory item for context t can store the task, support, certificates, outcome, and audit result:

$$M_t = r_{\text{task}} \otimes c_t \oplus r_{\text{support}} \otimes \sum_{m \in S_t} r_m \otimes H_m \quad (92)$$

$$\oplus r_{\text{certificate}} \otimes \sum_{m \in S_t} r_m \otimes Z_m \oplus r_{\text{outcome}} \otimes y_t \oplus r_{\text{audit}} \otimes a_t. \quad (93)$$

Future contexts can retrieve similar memories by HDC similarity, warm-start the support controller, propose one-swap candidates, avoid overused columns, or suggest shell promotions and demotions.

9.8 Option memory for RL and agentic systems

In reinforcement learning, a column may encode a reusable option, skill, or world-model fragment rather than a perceptual motif. HDC gives a role-filler format for option memory:

$$H_{\text{option}} = r_{\text{pre}} \otimes H_{\text{pre}} \oplus r_{\text{act}} \otimes H_{\text{act}} \oplus r_{\text{eff}} \otimes H_{\text{eff}} \oplus r_{\text{value}} \otimes H_V \oplus r_{\text{uncertainty}} \otimes H_U. \quad (94)$$

The controller can query by current-state preconditions or desired effects to retrieve candidate options. This is a natural extension of HDC-guided factorization from perceptual memory to agentic action memory.

9.9 A ColBaC-HDC error decomposition

Let $\mathcal{L}_{\text{ColBaC}}$ be the task loss of the full ColBaC model, $\mathcal{L}_{\text{fact}}$ the loss when each active column is replaced by its factorized column graph, and $\mathcal{L}_{\text{HDC-ColBaC}}$ the loss after HDC encoding, retrieval, and cleanup. Under HBCML adequacy errors $(\eta_{\text{app}}, \eta_g, \eta_h)$, HDC factorization defect δ_Q , binding distortion η_{bind} , and local cleanup loads k_m , a schematic bound is

$$\mathcal{L}_{\text{HDC-ColBaC}} \leq \mathcal{L}_{\text{ColBaC}} + C_1 \delta_Q + C_2 \sum_{m \in S_t} M_m \exp(-cD/k_m) + C_3 \eta_{\text{bind}}. \quad (95)$$

Similarly, the effective forgetting or update-commutator bound receives an additive representation term:

$$\|[\tilde{X}_i, \tilde{X}_j]\| \leq C_H(\eta_h G_{\max} + \eta_g H_{\max} + \eta_{\text{app}}) + C_{\text{HDC}} \left(\delta_Q + e^{-cD/k} + \eta_{\text{bind}} \right). \quad (96)$$

The constants and norms depend on the task model. The conceptual point is the three-way separation:

$$\text{continual-learning error} = \text{HBCML modularity error} + \text{factorization defect} + \text{HDC coding error}. \quad (97)$$

9.10 Why this is more general than ColBaC

ColBaC makes the HDC-guided idea especially direct because columns, shells, certificates, and one-swap teachers already provide the right hooks. But the principle is broader. Any algorithm with internal structure aligned to the data and tasks can use HDC requirements as a pressure on its own state: keep local interaction order small, keep role-bound factors stable, make codebooks well-separated, expose certificates, and ensure that task-relevant residuals are small.

10 Experimental program

10.1 Synthetic calibration

E1: single-step capacity. Bundle k random atoms, unbind a target, clean up against a codebook of size M , and fit the boundary $k \approx \alpha D / \log M$.

E2: descent with explicit subtree dictionaries. Generate bounded-branching trees with known $\mathcal{K}_t^{\text{sub}}$. Vary N by increasing depth while keeping M_t and k_t fixed. For a fixed query family, measure the D required to maintain a target error. The predicted curve is flat except for the explicit depth factor.

E3: all-path negative control. At the same D , query every leaf. Global reconstruction should fail as N grows unless $Dq_H = \Omega(N \log m)$. This experiment verifies that the method is not secretly violating the entropy bound.

E4: resonator margin. Measure convergence as a function of factor codebook sizes, local load, initialization, damping, projection, and margin. Report recomposition scores for successes and spurious fixed points.

E5: ablations. Remove cleanup, remove ordered roles, remove level codebooks, replace structured latents by random nonfactorized latents, and run a nondecomposable task such as global parity. Each ablation should destroy the corresponding theorem condition.

10.2 DNN latent experiments

Video. Use a pretrained or trained video encoder that yields object slots, VQ tokens, or latent events. Build a factor graph over object identity, position, velocity, relations, and time. Test object tracking, event retrieval, action classification, and anomaly detection. Plot task accuracy versus D , cleanup failure versus local load, and the full-DNN-minus-HDC accuracy gap as an empirical estimate of δ_Q .

Images. Use multiscale VQ tokens, patches, scene graphs, or slots. Test object/property queries, localized retrieval, and scene classification. Include a texture-heavy or global-style task as a high-defect control.

Language. Use token/span hierarchies, parse roles, retrieval chunks, or sparse-autoencoder features from a language model. Test entity retrieval, document routing, attribution, syntax/semantic probes, and structured QA over cached features. Do not claim faithful open-ended generation unless the defect is measured for that task.

Baselines. Compare against the full DNN latent readout, a compressed latent baseline such as PCA or product quantization, an indexed tree or ANN index, a raw latent scan, a nonhierarchical HDC bundle, and a no-cleanup HDC bundle. The HDC method should win only where local load is bounded and the query family matches the factorized structure.

10.3 HDC-guided learning experiments

Train an encoder with and without the HDC-guided losses of Section 7. Compare the curves for Δ_{fact} , Δ_{HDC} , k_{eff} , M_{conf} , cleanup margin, resonator success, and residual-probe accuracy. The prediction is not merely that HDC-guided training improves final accuracy. The sharper prediction is that it moves these intermediate diagnostics in the capacity-favorable direction.

A useful ablation sequence is:

1. train the full DNN task model;
2. add factorization but no HDC losses;
3. add margin and load losses;
4. add role-consistency and product-recomposition losses;
5. add the actual HDC encode/decode path in the task loop.

This separates whether failures come from the factor graph, the HDC geometry, or the task head.

10.4 R-HMH episodic memory and world-modeling experiments

A direct experiment for Section 8 is to compare flat episodic embeddings, ordinary HMH episodic keys, and R-HMH episodic keys with resonator cleanup. Useful task families include partial cue completion, factor-specific retrieval, counterfactual factor swaps, affordance-trial recall, social-script retrieval, and schema consolidation. Report whole-episode ANN accuracy separately from slot recovery accuracy, because a system can retrieve a roughly similar episode while still failing to recover the causal or social factor that matters for the query.

A minimal benchmark should track

$$\Delta_{\text{fact}}^{\text{epis}}, \quad \Delta_{\text{HDC}}^{\text{epis}}, \quad k_{\text{slot}}, \quad k_{\text{rel}}, \quad M_{\text{conf}}, \quad \Delta_{\text{cleanup}}, \quad p_{\text{res}}, \quad (98)$$

plus downstream judgment accuracy for neural models trained on raw dense embeddings versus $\mathcal{D}_{\text{hnh}}(H_E)$. The expected win is not perfect replay of all perceptual detail, but improved factor-aware recall and better downstream judgment when the task depends on the structure of the episode.

10.5 ColBaC-HDC experiments

For ColBaC/HBCML, compare four systems:

$$\text{A: ColBaC baseline,} \quad \text{B: ColBaC + HDC certificates,} \quad \text{C: ColBaC + HDC factorization losses,} \quad \text{D: ColBaC +} \quad (99)$$

The most likely early win is not raw benchmark accuracy but improved selector geometry: lower selector regret, better one-swap warm starts, less support churn, better prediction of audited support quality, and more stable shell promotion. Report

$$\mathcal{L}_{\text{full ColBaC}}, \quad \mathcal{L}_{\text{factorized ColBaC}}, \quad \mathcal{L}_{\text{HDC-ColBaC}}, \quad (100)$$

plus support rank, one-swap recovery, support entropy, support churn, promotion precision, cross-column HDC overlap, certificate-predicted audit gain, and forgetting. If HDC certificates are useful, they should improve retrieval of good support neighborhoods before the one-swap teacher acts.

11 Application regimes

Video is arguably the strongest first DNN target because object persistence and time-local dynamics naturally induce bounded local structure. Language is also interesting, especially for retrieval and mechanistic features, but it requires careful task restriction. Static images are a useful intermediate case when the encoder provides multiscale VQ tokens, slots, or scene graphs. Columnar causal continual-learning systems are a particularly strong architectural target because their supports, columns, shells, and certificates already mirror HDC roles, codebooks, and local-load budgets.

Domain	Learned factors	Good queries	Main risk
Video	objects, time, pose, velocity, relations, events	tracking, event retrieval, action, anomaly	occlusion and global context
Images	patches, VQ tokens, slots, scene graph nodes	object/property lookup, scene classification, localized retrieval	texture/style defect
Language	tokens, spans, entities, syntax roles, sparse LM features	retrieval, probes, structured QA, routing	long-range pragmatics and generation defect
Causal latents	mechanisms, interventions, parents, state transitions	counterfactual lookup, planning features, invariant prediction	identifiability assumptions
ColBaC/HBCML	columns, shells, microcolumns, certificates, supports	support retrieval, shell hygiene, continual reuse, option memory	premature discretization and dense bridge load
Episodic world models	actors, roles, goals, affect, affordances, evidence anchors, causal and temporal links	partial recall, slot completion, counterfactual judgment, schema consolidation	holistic context defect and unbounded relation load
Relational data	schema roles, records, keys, foreign-key trees	lookup, aggregation, compression	cross-edge load

Table 1: Natural regimes for learned factor graph HDC codes. The method is best viewed as a task-preserving cache of structured latents.

12 Discussion

What the theory says. The dimension needed for local query fidelity is controlled by local load, effective codebook size, cleanup margin, binding distortion, depth, supported query count, and search budget. It is not controlled by raw data size except through those quantities. This is the clean version of the intuition that hierarchical factorization prevents every query from paying for every leaf.

What the theory does not say. It does not say that one vector losslessly stores arbitrary images, videos, documents, or agent states. It does not say that resonators converge from any initialization. It does not say that arbitrary root cosine similarity computes arbitrary tree statistics. It does not say that HDC local lookup beats a well-engineered indexed tree asymptotically. It says that fixed-width HDC can be an efficient task-preserving cache when the upstream representation has low defect, low local load, good margins, and reusable state dictionaries.

The HDC layer as a learning guide. The most interesting idea in this paper is that the HDC layer is not merely a post-hoc compressor. Its requirements can be fed back into learning. It tells the upstream system which factorizations are useful: those that make each supported query depend on a small number of well-separated role-bound factors, with little task-relevant information left outside the factor graph. This is a different objective from generic disentanglement or reconstruction. It is capacity-aware, query-relative, and operational.

R-HMH as episodic memory. The episodic-memory extension makes the same point in cognitive terms. Episodes are useful to the extent that the agent can recall and recombine their roles, causes, affordances, outcomes, and evidence anchors. R-HMH stores an episode as a factorized modular hypervector graph rather than as a flat trace, and resonator cleanup supplies a mechanism for partial completion of missing roles or product factors. The price is the same as elsewhere: if

the supported episodic queries require holistic context not captured by bounded-load factors, the factorization defect dominates.

ColBaC as a natural host. ColBaC already contains sparse support selection, typed columns, reusable shells, internal certificates, and local teachers. HDC adds an algebraic memory and query language for these components. In this setting, HDC can regularize column contents, enrich certificates, warm-start support selection, guide shell promotion, detect bridge overload, and store episodic support memories. The architecture therefore provides a concrete testbed for HDC-guided factorization, not just an application domain.

Relation to rate-distortion-compute. The code lies on a rate-distortion-compute surface. Smaller D reduces storage and nominal HDC compute, but increases cleanup and factorization error. Once D exceeds the local capacity threshold, the remaining loss is the task-specific factorization defect. The decisive empirical plot is task accuracy versus D , with the saturation floor compared to the full neural or algorithmic state.

Best near-term claim. A conservative and useful thesis is:

Structured learners expose or learn reusable local factors; resonator-HDC codes provide a compact, compositional, queryable cache of those factors; and the HDC capacity law can guide the learner toward factorizations that are cheaper to store, retrieve, audit, and reuse.

This claim applies to DNNs, ColBaC-style causal columns, symbolic algorithms, program analyzers, database engines, and agentic option memories whenever their internal structure is aligned with the dataset and task structure.

13 Conclusion

We presented a query-limited theory of resonator-factored hierarchical hypervector codes for approximately factorized learned structures. The framework uses explicit or factored level dictionaries, ordered roles, task-specific factorization defect, conditional resonator margins, quasi-isometric binding requirements, and honest storage and compute accounting. These restrictions are not incidental. They are the conditions under which an HDC code can be a compact and efficient surrogate for a structured latent graph without violating entropy bounds or hiding search costs.

The paper’s main conceptual move is to make the HDC layer bidirectional. It is not only a cache placed after a DNN or structured algorithm. It is also a source of pressure on the upstream learner. The HDC capacity law says what kind of factorization is valuable: low-load, role-stable, high-margin, low-confusability, product-recoverable, intervention-local factors that preserve the supported queries and leave little task-relevant information in the residual. These requirements can be turned into losses, audits, and live diagnostics.

The same bidirectional move applies to episodic memory and world modeling. By combining resonator-HDC with HMM, an episode can be represented as a resonantly decodable modular factor graph rather than as a monolithic remembered vector. Actors, objects, actions, goals, places, times, affective states, affordance contexts, causal links, outcomes, evidence anchors, and invoked skills can be stored in named ontology-aligned slots. Partial cues can then retrieve episodes, resonator cleanup can complete missing factors, counterfactual factor swaps can train neural judgment models, and

repeated retrieval can consolidate experiences into semantic schemas, affordance templates, social scripts, and options.

The ColBaC/HBCML instantiation makes this especially concrete. HBCML supplies a two-level modular learning substrate with sparse supports, internal probabilistic states, certificates, and local teachers. ColBaC supplies columns, typed microcolumns, hard kernels, adaptable shells, and support audits. HDC turns these into addressable role-filler structures: column codes, certificate hypervectors, support memories, option memories, R-HMH episodic traces, and shell-hygiene criteria. The result is a plausible synthesis: ColBaC learns and protects causal modules, while HDC makes their reusable contents compact, compositional, queryable, auditable, and capacity-aware.

The decisive empirical question is therefore not whether natural data is exactly factorized. It is whether a learner can expose a low-defect, bounded-load factor graph for the tasks one wants to answer, and whether HDC-guided learning makes that factor graph easier to store, query, reuse, and protect over time.

References

- [1] P. Kanerva. Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors. *Cognitive Computation*, 1:139–159, 2009.
- [2] T. A. Plate. Holographic reduced representations. *IEEE Transactions on Neural Networks*, 6(3):623–641, 1995.
- [3] D. Kleyko, M. Davies, E. P. Frady, P. Kanerva, S. J. Kent, B. A. Olshausen, E. Osipov, J. M. Rabaey, D. A. Rachkovskij, A. Rahimi, and F. T. Sommer. Vector symbolic architectures as a computing framework for emerging hardware. *Proceedings of the IEEE*, 2022.
- [4] E. P. Frady, S. J. Kent, B. A. Olshausen, and F. T. Sommer. Resonator networks for factoring distributed representations of data structures. *Neural Computation*, 32(12):2311–2331, 2020.
- [5] Y. Bengio, A. Courville, and P. Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.
- [6] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. *Advances in Neural Information Processing Systems*, 2017.
- [7] A. Razavi, A. van den Oord, and O. Vinyals. Generating diverse high-fidelity images with VQ-VAE-2. *Advances in Neural Information Processing Systems*, 2019.
- [8] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf. Object-centric learning with slot attention. *Advances in Neural Information Processing Systems*, 2020.
- [9] F. Locatello, S. Bauer, M. Lucic, G. Raetsch, S. Gelly, B. Schoelkopf, and O. Bachem. Challenging common assumptions in the unsupervised learning of disentangled representations. *International Conference on Machine Learning*, 2019.
- [10] J. Brehmer, P. de Haan, P. Lippe, and T. Cohen. Weakly supervised causal representation learning. *Advances in Neural Information Processing Systems*, 2022.

- [11] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv:2309.08600*, 2023.
- [12] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- [13] J. Snider and S. Franklin. Modular composite representation. *Cognitive Computation*, 6(3):510–527, 2014.
- [14] B. Goertzel. Hierarchical Modular Hypervectors: Baking type-based hierarchical embeddings into modular compositional hypervectors. *Manuscript*, 2025.
- [15] B. Goertzel. PRIMUS world modeling with symbolic, HMH, and dense neural representations. *Manuscript*, 2026.
- [16] B. Goertzel. Causal coding and the general causal-continual-learning theorem. *Manuscript*, 2025.
- [17] B. Goertzel. Toward a general theory of hierarchical Bayesian causal modular learning. *Manuscript*, 2026.
- [18] B. Goertzel. Columnar Bayesian causal coding networks: A two-level architecture for causal modularity, continual learning, universality, and wavelet-transformer integration. *Manuscript*, 2026.
- [19] B. Goertzel. Teacher-first columnar control for continual learning: A two-level architecture with exact support search. *Manuscript*, 2026.
- [20] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- [21] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*, 2017.
- [22] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [23] A. A. Rusu, N. C. Rabinowitz, G. Desjardins, H. Soyer, J. Kirkpatrick, K. Kavukcuoglu, R. Pascanu, and R. Hadsell. Progressive neural networks. *arXiv:1606.04671*, 2016.